

SDN-Lab1

Haochen You

March 2026

1 Statement of Work

Warm-up

- Try Mininet CLI to create and interact with a network
- Try ovs-vsctl to interact with Open vSwitch (OVS)
- Use wireshark to capture packets in Mininet

Task

- Construct a fattree (k=4) topology using Mininet Python API
- Make sure all hosts are reachable
- If not reachable, try to solve it
- If reachable, show the path between two host

2 Warm-up

2.1 Create and Interact with a Network

如下图所示，使用 `sudo mn` 命令可以创建一个默认的基础网络拓扑，其中包含两个主机和一个交换机。接下来我们执行 `nodes`, `links` 等命令可以查看这个网络相关的信息，还可以使用 `pingall` 来测试网路的连通性，最后，我们退出并清理环境。

```
[sdn@archlinux ~]$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> nodes
available nodes are:
c0 h1 h2 s1
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s1-eth2 (OK OK)
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0
c0
```

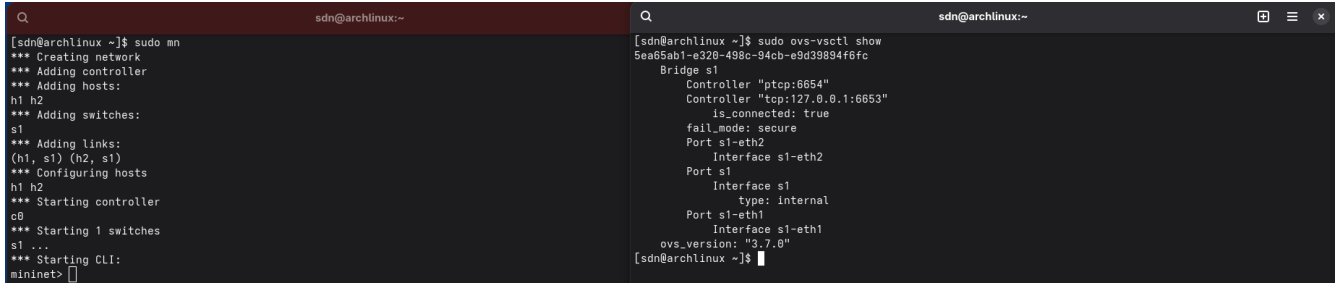
图 1: Basic Instructions

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet> sudo mn -c
*** Unknown command: sudo mn -c
mininet> exit
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 556.523 seconds
[sdn@archlinux ~]$ sudo mn -c
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflow ovs-controller ovs-testcon
troller udgbwtest mnexec ivs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflow ovs-controller ovs-test
controller udgbwtest mnexec ivs ryu-manager 2> /dev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old xll tunnels
*** Removing excess kernel datapaths
```

图 2: Basic Instructions

2.2 Interact with OVS

Mininet 底层实际上是在用 OVS 作为虚拟交换机，下面我们在一个终端上启动基础网络，然后打开一个新终端，输入 `sudo ovs-vsctl show` 即可以观察到交换机的状态，如下图所示：

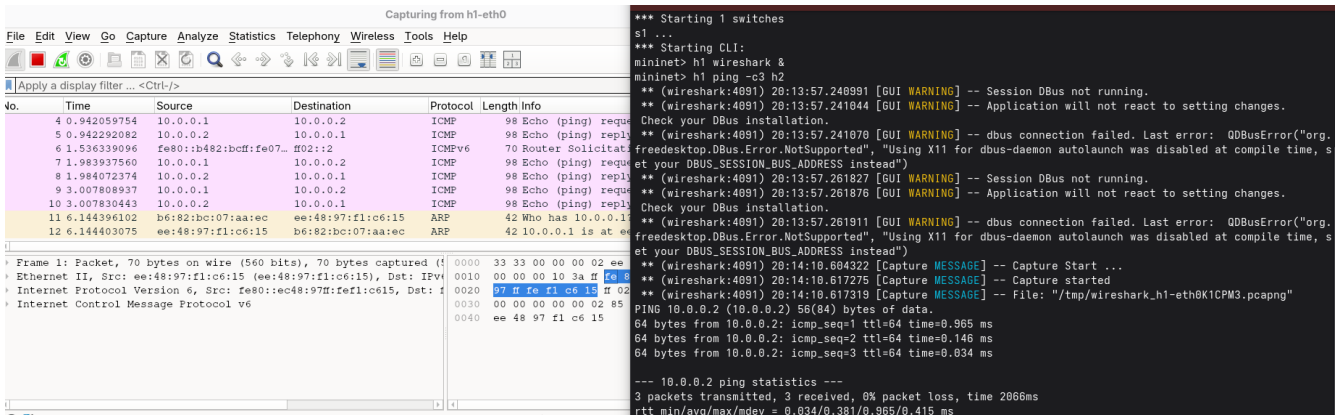


The image shows two terminal windows. The left window shows the output of `sudo mn` in a Mininet environment, which creates a network with two hosts (h1, h2) and one switch (s1). The right window shows the output of `sudo ovs-vsctl show`, displaying the configuration of the Open vSwitch (s1), including its controller (tcp:127.0.0.1:6653), interfaces (s1-eth2), and ports (s1, s1-eth1).

图 3: Interact with OVS

2.3 Use Wireshark to Capture the Packets

在命令行执行命令在后台运行 WireShark，用 h1 主机 ping h2，这样我们在 WireShark 能看到抓到的 ICMP 请求和回复包，如下图所示：



The image shows two windows. The left window is the Wireshark interface, capturing traffic on the h1-eth0 interface. It displays a list of captured packets, including ICMP Echo (ping) requests and replies. The right window is a terminal showing the Mininet setup and the execution of `h1 ping -c3 h2`, which triggers the capture of ICMP packets in Wireshark.

图 4: WireShark Capture

3 Tasks

3.1 Task1:Construct a Fattree Topology

在 $k=4$ 的 fattree topology 中，网络分为 Core 核心层，Aggregation 汇聚层，Edge 边缘层三个层级。根据公式计算，我们算出来该拓扑需要 16 台主机，8 个汇聚交换机，8 个边缘交换机，4 个核心交换机。接下来我们使用推荐的 Python API 直接创建并运行网络的方式编写脚本来实现：但在实现之前，我们先需要使用虚拟机的 SSH 服务器，远程连接 VSCode 使得我们有可用的开发环境。

第一步，我们在 VSCode 下载 Remote-SSH 插件。第二步，在虚拟机的终端输入命令 `ip a`，看看虚拟机使用的是默认的 NAT 网络还是桥接网络，本机使用的是默认的 NAT 网络，所以需要进行端口转发。第三步，在 VirtualBox 管理器，设置 SDN 虚拟机的端口转发。配置好之后，在 VSCode 中连接，我们就能看到如下图所示的页面：

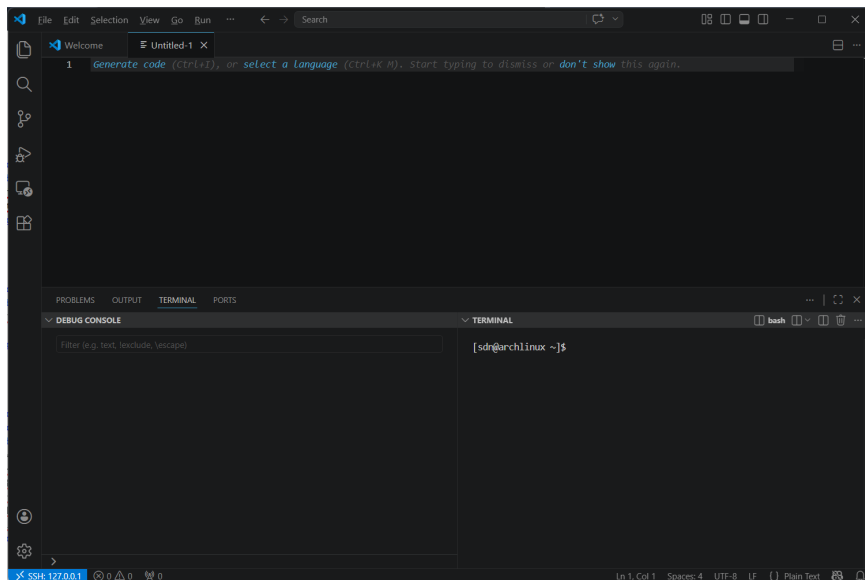


图 5: VSCode SSH

配置完之后，我们在 VSCode 上编写程序 `fattree.py`，保存在虚拟机上，在命令行执行 `sudo python fattree.py`，执行结果如下图所示：

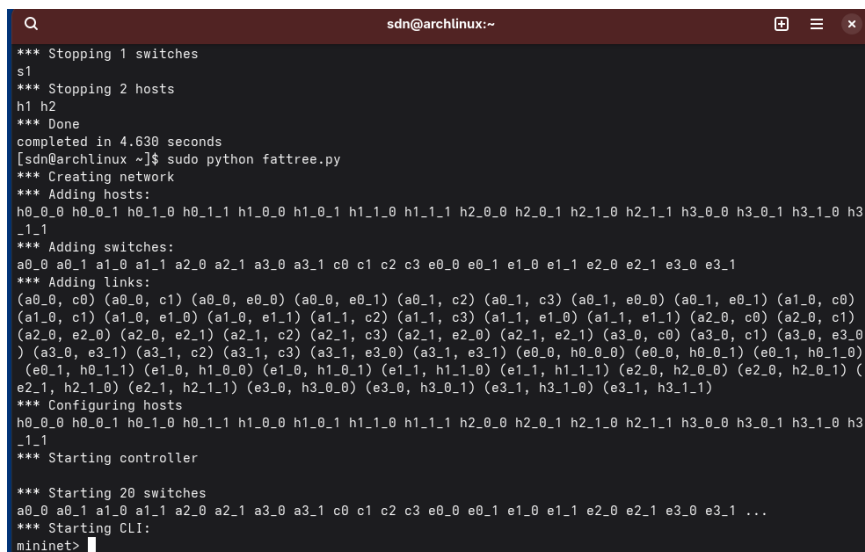


图 6: Fat Tree Building Result

在输出信息中我们可以看到，Adding hosts 列出了一共 16 台主机，Adding switches 列出了所有交换机（其中带 a 的汇聚 8 个，带 e 的边缘 8 个，带 c 的核心 4 个），而且 Starting Controller 下面是空行，说明我

们成功屏蔽了控制器。这样，我们就构建了 $k=4$ 的 fattree 拓扑。

下面我们给出 fattree.py 的源码：

```
1 from mininet.topo import Topo
2 from mininet.net import Mininet
3 from mininet.cli import CLI
4 from mininet.log import setLogLevel
5
6 class FatTree(Topo):
7     def build(self):
8         k = 4
9         cores = [self.addSwitch(f'c{i}') for i in range(4)]
10
11         for p in range(k):
12             aggs = [self.addSwitch(f'a{p}_{i}') for i in range(k//2)]
13             edges = [self.addSwitch(f'e{p}_{i}') for i in range(k//2)]
14
15             for a in aggs:
16                 for e in edges:
17                     self.addLink(a, e)
18
19             for i, e in enumerate(edges):
20                 for j in range(k//2):
21                     host = self.addHost(f'h{p}_{i}_{j}')
22                     self.addLink(e, host)
23
24             for i, a in enumerate(aggs):
25                 for j in range(k//2):
26                     core_idx = i * (k//2) + j
27                     self.addLink(a, cores[core_idx])
28
29 if __name__ == '__main__':
30     setLogLevel('info')
31     topo = FatTree()
32     net = Mininet(topo=topo, controller=None)
33     net.start()
34     CLI(net)
35     net.stop()
```

3.2 Task2:Test if All Hosts are Reachable

接下来我们执行 pingall 命令，测试连通性，结果如下图所示：

```

mininet> pingall
*** Ping: testing ping reachability
h0_0_0 -> X X X X X X X X X X X X X X
h0_0_1 -> X X X X X X X X X X X X X X
h0_1_0 -> X X X X X X X X X X X X X X
h0_1_1 -> X X X X X X X X X X X X X X
h1_0_0 -> X X X X X X X X X X X X X X
h1_0_1 -> X X X X X X X X X X X X X X
h1_1_0 -> X X X X X X X X X X X X X X
h1_1_1 -> X X X X X X X X X X X X X X
h2_0_0 -> X X X X X X X X X X X X X X
h2_0_1 -> X X X X X X X X X X X X X X
h2_1_0 -> X X X X X X X X X X X X X X
h2_1_1 -> X X X X X X X X X X X X X X
h3_0_0 -> X X X X X X X X X X X X X X
h3_0_1 -> X X X X X X X X X X X X X X
h3_1_0 -> X X X X X X X X X X X X X X
h3_1_1 -> X X X X X X X X X X X X X X
*** Results: 100% dropped (0/240 received)

```

图 7: Pingall Result

在图中我们观察到了满屏的 X，这说明现在任何两台主机之间都无法通信。

3.3 Task3:Find the Issues and Solve It

经过查阅资料，我分析了主机之间无法通信的原因：从某种意义上说，这正是实验要求的“不使用控制器”造成的。当正常使用 Mininet 启动默认网络时，会自动带有一个控制器。当交换机受到未知的数据包，他会询问控制器处理方法，控制器会告诉交换机从哪个端口去目标主机。现在我们没有使用控制器，这些由 OVS 模拟出来的虚拟交换机都相当于处于出厂模式，他们的 MAC 地址表和流表都是空的。当主机试图给别的机器发包，它连接的第一个边缘交换机收到了包，完全不知道往哪个端口转发。

接下来我们尝试解决这个问题。首先我们关闭 OpenFlow 的 Secure 模式（这个模式是因为我们关闭了 controller 才出现的），在这种模式之下，交换机遵守“没有流表就不转发”的原则。其次，就算我们恢复了自学习模式，主机之间也是无法 ping 通的，这是因为 fat tree 拓扑产生了大量的物理环路。在这个过程之中，主机因为首次通信，必须发送 ARP 广播包来询问目标 IP 的 MAC 地址。在传统的自学习交换机中，主机遇到广播包会向除接收端口外的所有端口进行 Flooding 洪泛，在有环路的 fat tree 拓扑之中，网络中瞬间产生指数级增长的广播风暴，导致正常的 ping 是无法 ping 通的。我们的解决方案是开启 STP 生成树协议来斩断环路（通过 switch=partial(OVSBridge, stp=True) 参数），这使得最初的 ARP 广播包能够安全地洪泛全网并得到回应，而不会引发广播风暴。

下面的图片展示了截取到的“广播风暴”：

No.	Time	Source	Destination	Protocol	Length	Info
5	16.44855286	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
6	17.47972927	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
7	18.60520308	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
8	19.65504894	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
9	20.67564436	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
10	21.69803798	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
11	22.72707221	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
12	23.74414544	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
13	24.76772154	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
14	25.83365851	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
15	26.89050639	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
16	27.90305142	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
17	28.93242592	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
18	29.95305488	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
19	30.98031132	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
20	32.10053639	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
21	33.10230460	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
22	34.10478609	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
23	35.10716716	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
24	36.10971096	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
25	37.11247450	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
26	38.11513511	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
27	39.12235516	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
28	40.12563433	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
29	41.12808533	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
30	42.13043355	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
31	43.13585320	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
32	43.13520474	78:01:1097:169f:fa3::02:1:2	IGMPv6	70	Router Solicitation from 02:1e:f6:9a:31:24:3f	
33	44.14030546	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
34	45.14060302	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
35	46.14051802	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
36	47.14077236	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
37	48.15220509	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
38	49.15300175	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
39	50.15601518	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
40	51.15848350	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
41	52.16090253	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
42	53.16319759	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
43	54.16680099	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
44	55.16845484	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
45	56.17072820	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
46	57.17280176	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
47	58.17694943	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
48	59.17756640	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
49	60.17998118	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1
50	61.18593389	02:7f:69:a3:124:3f	Broadcast	ARP	42	Who has 10.0.0.254? Tell 10.0.0.1

Q

(v4search:17674) 22:16:26.314032 [Capture MSSQL]

PING 10.0.0.254 (10.0.0.254) 56(0.4) bytes of data:

From 10.0.0.1 icmp_seq=4 Destination Host unreachable

From 10.0.0.1 icmp_seq=5 Destination Host unreachable

From 10.0.0.1 icmp_seq=6 Destination Host unreachable

From 10.0.0.1 icmp_seq=7 Destination Host unreachable

From 10.0.0.1 icmp_seq=8 Destination Host unreachable

From 10.0.0.1 icmp_seq=9 Destination Host unreachable

From 10.0.0.1 icmp_seq=10 Destination Host unreachable

From 10.0.0.1 icmp_seq=11 Destination Host unreachable

From 10.0.0.1 icmp_seq=12 Destination Host unreachable

From 10.0.0.1 icmp_seq=13 Destination Host unreachable

From 10.0.0.1 icmp_seq=14 Destination Host unreachable

From 10.0.0.1 icmp_seq=15 Destination Host unreachable

From 10.0.0.1 icmp_seq=16 Destination Host unreachable

From 10.0.0.1 icmp_seq=17 Destination Host unreachable

From 10.0.0.1 icmp_seq=18 Destination Host unreachable

From 10.0.0.1 icmp_seq=19 Destination Host unreachable

From 10.0.0.1 icmp_seq=20 Destination Host unreachable

From 10.0.0.1 icmp_seq=21 Destination Host unreachable

From 10.0.0.1 icmp_seq=22 Destination Host unreachable

From 10.0.0.1 icmp_seq=23 Destination Host unreachable

From 10.0.0.1 icmp_seq=24 Destination Host unreachable

From 10.0.0.1 icmp_seq=25 Destination Host unreachable

From 10.0.0.1 icmp_seq=26 Destination Host unreachable

From 10.0.0.1 icmp_seq=27 Destination Host unreachable

From 10.0.0.1 icmp_seq=28 Destination Host unreachable

From 10.0.0.1 icmp_seq=29 Destination Host unreachable

From 10.0.0.1 icmp_seq=30 Destination Host unreachable

From 10.0.0.1 icmp_seq=31 Destination Host unreachable

From 10.0.0.1 icmp_seq=32 Destination Host unreachable

From 10.0.0.1 icmp_seq=33 Destination Host unreachable

From 10.0.0.1 icmp_seq=34 Destination Host unreachable

From 10.0.0.1 icmp_seq=35 Destination Host unreachable

From 10.0.0.1 icmp_seq=36 Destination Host unreachable

From 10.0.0.1 icmp_seq=37 Destination Host unreachable

From 10.0.0.1 icmp_seq=38 Destination Host unreachable

From 10.0.0.1 icmp_seq=39 Destination Host unreachable

图 8: Broadcast Storm

接下来我们修复这个问题，在代码中关闭所有的 Secure 模式，之后开启 STP 生成树协议。代码如下：

```

1 from functools import partial
2 from mininet.topo import Topo
3 from mininet.net import Mininet
4 from mininet.cli import CLI
5 from mininet.log import setLogLevel
6 from mininet.node import OVSBridge
7
8 class FatTree(Topo):
9     def build(self):
10         k = 4
11         cores = [self.addSwitch(f'c{i}') for i in range(4)]
12
13         for p in range(k):
14             aggs = [self.addSwitch(f'a{p}_{i}') for i in range(k//2)]
15             edges = [self.addSwitch(f'e{p}_{i}') for i in range(k//2)]
16
17             for a in aggs:
18                 for e in edges:
19                     self.addLink(a, e)
20
21             for i, e in enumerate(edges):
22                 for j in range(k//2):
23                     host = self.addHost(f'h{p}_{i}_{j}')
24                     self.addLink(e, host)
25
26             for i, a in enumerate(aggs):
27                 for j in range(k//2):
28                     core_idx = i * (k//2) + j
29                     self.addLink(a, cores[core_idx])
30
31 if __name__ == '__main__':
32     setLogLevel('info')
33     topo = FatTree()
34     net = Mininet(topo=topo, controller=None, switch=partial(OVSBridge, stp=True), autoSetMacs=True,

```

```

    waitConnected=True)
35 net.start()
36
37 for switch in net.switches:
38     switch.cmd('ovs-vsctl del-fail-mode {switch.name}')
39
40 CLI(net)
41 net.stop()

```

在完成这一步之后，我们再来执行 pingall 命令，发现主机之间都能 ping 通了。

```

[sdn@archlinux ~]$ sudo python updated.py
*** Creating network
*** Adding hosts:
h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
*** Adding switches:
a0_0 a0_1 a1_0 a1_1 a2_0 a2_1 a3_0 a3_1 c0 c1 c2 c3 e0_0 e0_1 e1_0 e1_1 e2_0 e2_1 e3_0 e3_1
*** Adding links:
(a0_0, c0) (a0_0, c1) (a0_0, e0_0) (a0_0, e0_1) (a0_1, c2) (a0_1, c3) (a0_1, e0_0) (a0_1, e0_1) (a1_0, c0) (a1_0, c1) (a1_0, e1_0) (a1_0, e1_1) (a1_1, c2) (a1_1, c3) (a1_1, e1_0) (a1_1, e1_1) (a2_0, c0) (a2_0, c1) (a2_0, e2_0) (a2_0, e2_1) (a2_1, c2) (a2_1, c3) (a2_1, e2_0) (a2_1, e2_1) (a3_0, c0) (a3_0, c1) (a3_0, e3_0) (a3_0, e3_1) (a3_1, c2) (a3_1, c3) (a3_1, e3_0) (a3_1, e3_1) (e0_0, h0_0_0) (e0_0, h0_0_1) (e0_1, h0_1_0) (e0_1, h0_1_1) (e1_0, h1_0_0) (e1_0, h1_0_1) (e1_1, h1_1_0) (e1_1, h1_1_1) (e2_0, h2_0_0) (e2_0, h2_0_1) (e2_1, h2_1_0) (e2_1, h2_1_1) (e3_0, h3_0_0) (e3_0, h3_0_1) (e3_1, h3_1_0) (e3_1, h3_1_1)
*** Configuring hosts
h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
*** Starting controller

*** Starting 20 switches
a0_0 a0_1 a1_0 a1_1 a2_0 a2_1 a3_0 a3_1 c0 c1 c2 c3 e0_0 e0_1 e1_0 e1_1 e2_0 e2_1 e3_0 e3_1 ...
*** Waiting for switches to connect
a0_0 a1_0 a1_1 a2_0 a2_1 a3_0 a3_1 c0 c1 c2 c3 e0_0 e0_1 e1_0 e1_1 e2_0 e2_1 e3_0 e3_1 a0_1
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h0_0_0 -> h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h0_0_1 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h0_1_0 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h0_1_1 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h1_0_0 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h1_0_1 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h1_1_0 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h1_1_1 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h2_0_0 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h2_0_1 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h2_1_0 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h2_1_1 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h3_0_0 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h3_0_1 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h3_1_0 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
h3_1_1 -> h0_0_0 h0_0_1 h0_1_0 h0_1_1 h1_0_0 h1_0_1 h1_1_0 h1_1_1 h2_0_0 h2_0_1 h2_1_0 h2_1_1 h3_0_0 h3_0_1 h3_1_0 h3_1_1
*** Results: 0% dropped (240/240 received)

```

图 9: Ping Result After Fixing

最后，我们查看各个设备的 MAC 地址：

```

mininet> py [print(h.name, h.MAC()) for h in net.hosts]
h0_0_0 00:00:00:00:00:01
h0_0_1 00:00:00:00:00:02
h0_1_0 00:00:00:00:00:03
h0_1_1 00:00:00:00:00:04
h1_0_0 00:00:00:00:00:05
h1_0_1 00:00:00:00:00:06
h1_1_0 00:00:00:00:00:07
h1_1_1 00:00:00:00:00:08
h2_0_0 00:00:00:00:00:09
h2_0_1 00:00:00:00:00:0a
h2_1_0 00:00:00:00:00:0b
h2_1_1 00:00:00:00:00:0c
h3_0_0 00:00:00:00:00:0d
h3_0_1 00:00:00:00:00:0e
h3_1_0 00:00:00:00:00:0f
h3_1_1 00:00:00:00:00:10
[None, None, None, None, None, None, None, None, None, None, None, None, None, None, None]

```

图 10: MAC Address

3.4 Task4: Transmission Analysis

下面我们由 h0_0_0 来 ping h3_1_1，顺带一路显示转发表，看看这个过程中是如何执行的，下面是结果：

```
mininet> h0_0_0 ping -c 1 h3_1_1
PING 10.0.0.16 (10.0.0.16) 56(84) bytes of data.
64 bytes from 10.0.0.16: icmp_seq=1 ttl=64 time=0.729 ms

--- 10.0.0.16 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.729/0.729/0.729/0.000 ms
mininet> sh sudo ovs-appctl fdb/show e0_0
port  VLAN  MAC  Age
3      0      00:00:00:00:00:01  7
2      0      00:00:00:00:00:10  7
mininet> sh sudo ovs-appctl fdb/show a0_1
port  VLAN  MAC  Age
1      0      00:00:00:00:00:01  12
3      0      00:00:00:00:00:10  12
mininet> sh sudo ovs-appctl fdb/show c2
port  VLAN  MAC  Age
1      0      00:00:00:00:00:01  16
4      0      00:00:00:00:00:10  16
mininet> sh sudo ovs-appctl fdb/show a3_1
port  VLAN  MAC  Age
3      0      00:00:00:00:00:01  20
2      0      00:00:00:00:00:10  20
mininet> sh sudo ovs-appctl fdb/show e3_1
port  VLAN  MAC  Age
2      0      00:00:00:00:00:01  25
4      0      00:00:00:00:00:10  25
```

图 11: Find the Route

我们分析一下通信的流程，在本次测试中，数据包从源主机 h0_0_0 出发，经历了一次完整的拓扑转发过程。首先，数据包进入其直连的边缘交换机 e0_0，经查表后被向上转发至汇聚交换机 a0_1。接着，a0_1 将其继续向上传递给负责跨区域通信的核心交换机 c2。作为全网枢纽，c2 识别出目标所在的 Pod 3，并将其向下投递至目标区域的汇聚交换机 a3_1。最后，数据包继续传送至目标边缘交换机 e3_1，由其直接派送到最终目标主机 h3_1_1。这一连串基于 MAC 端口转发路径，与我们在代码中构建的拓扑连线相符，验证了整个底层网络的连通性与逻辑。